Leakage-Aware Machine Learning Integration in a Multi-Tenant ERP Platform: A Case-Study-Driven Approach

[Author Name]
[Department / Institution]
[City, Country]
[email@domain]

***Abstract*—Embedding machine learning (ML) inside enterprise resource planning (ERP) systems is increasingly common, but published descriptions rarely address four practical hazards together: (i) target leakage that produces unrealistically optimistic metrics, (ii) per-tenant model lifecycle in shared-nothing multi-tenant deployments, (iii) heavy class imbalance in labels such as late payment or attrition, and (iv) distribution drift between training and inference. We describe an end-to-end ML pipeline integrated into a Django multi-tenant ERP, covering CRM lead scoring, HR attrition prediction, accounts-receivable late-payment risk, and inventory demand forecasting. The pipeline is evaluated on two real public datasets (Olist Brazilian e-commerce, $\sim$99,000 orders; IBM HR Analytics, 1,470 employees) and two synthetic generators where no comparable public ERP-grade data is available. We treat *leakage detection* as a first-class validation gate using a label-shuffle test, which rejected our initial lead-scoring features despite an apparently strong AUC of 0.9996. The same gate validated the corrected pipeline. We further present a detailed case study on late-payment prediction in which a naive delivery-delay-based label was replaced with a customer-level payer profile; the headline AUC fell from 0.63 to 0.50 before rising to 0.78 (5-fold cross-validation) once history-aware features became identifiable through customer bucketing. Beyond AUC, we report PR-AUC, F1 at a calibrated threshold, and precision@k to make imbalance-aware claims actionable. We describe a drift-monitoring stack (PSI, Kolmogorov–Smirnov, ADWIN, rolling realised performance), a tenant-level A/B-testing scaffold, and a four-stage roadmap to cross-tenant validation. We discuss what synthetic-data AUCs do and do not measure, the design of a per-tenant model registry, and the specific gap that must be closed before any of these numbers translates to production performance.**

***Index Terms*—machine learning, ERP, multi-tenant SaaS, target leakage, class imbalance, concept drift, A/B testing, demand forecasting, late-payment prediction, customer churn, employee attrition, MLOps**

I. Introduction

Modern ERP platforms manage operational data covering customers, invoices, products, employees, and inventory movements. In a multi-tenant software-as-a-service deployment a single instance of the application serves many independent customer organisations (*tenants*), each with an isolated data store, configuration, and—in our setting—an isolated machine-learning *model registry* (a per-tenant directory holding trained artefacts, metadata, and a training report) [1], [2]. Adding ML capabilities on top of this data is a natural step: lead scoring for sales pipelines, attrition risk for HR, late-payment risk for accounts receivable, and demand forecasting for inventory replenishment are all standard supervised or time-series problems with mature algorithms [3], [4].

In practice, however, integrating ML into a working multi-tenant ERP raises problems that the algorithmic literature largely sidesteps:

- **Target leakage from operational state.** ERP feature tables often contain post-event status fields (e.g., a lead's `won` flag, an invoice's `paid` timestamp). Naively including them in feature extraction yields models whose offline metrics are excellent and whose online performance collapses.
- **Per-tenant model lifecycle.** A shared-nothing multi-tenant SaaS deployment hosts each tenant's data in a separate database. A model trained on tenant A should never score tenant B's rows; the registry, feature pipeline, and serving layer must respect that boundary.
- **Sparse and imbalanced operational signals.** Most products sell rarely, most leads never convert, most employees do not leave, and most invoices are paid on time. Class imbalance and time-series sparsity are the rule, not the exception, and naive AUC-only evaluation can hide threshold-level failures [5], [6].
- **Distribution drift after deployment.** Tenants onboard new product lines, run promotions, and absorb macroeconomic shocks; an offline-perfect model can degrade silently within weeks once exposed to live traffic [7], [8].

This paper reports an end-to-end design that addresses each of the issues above, integrated into an open-source Django ERP we have developed. The contributions are:

1) A reproducible ML training pipeline integrated with a multi-tenant ORM and a per-tenant model registry (Section III).
2) A *shuffle-label* validation gate used to detect and reject leaky feature definitions, with a worked example showing how it caught leakage hidden behind AUC $\approx$ 1.0 (Section IV).
3) Four case studies—lead scoring, attrition, late-payment risk, and demand forecasting—each documenting the data, label, features, leakage check, and limitations (Sec-

tion IV-C). Two of the four use real public data (Olist e-commerce, IBM HR Analytics); the remaining two use synthetic generators because no comparable public ERP-grade dataset exists.

4) Imbalance-aware reporting (PR-AUC, F1 at a calibrated threshold, precision@k) and a class-weighting / SMOTE intervention applied *inside* the cross-validation loop (Section V).
5) A drift-monitoring stack combining feature-level Kolmogorov–Smirnov and PSI tests, ADWIN over the predicted-probability stream, and rolling realised-performance metrics, with a per-tenant alerting and shadow-mode promotion contract (Section VI).
6) A four-stage roadmap to real-world, multi-tenant validation (tenant-level A/B testing, champion-vs-challenger promotion, cross-tenant generalisation experiments, noise-injection audits) for which the present pipeline is a substrate (Section VII).
7) A discussion of what synthetic-data evaluation does and does not establish, and how to bridge the synthetic-to-real gap honestly (Section IX).

II. Related Work

ML for credit and payment risk. Lessmann et al. [3] benchmark 41 classifiers across 8 credit-scoring datasets, finding gradient boosting and random forests competitive with deep models, and report typical AUCs between 0.74 and 0.88 on real data. We use this range as a plausibility band for our late-payment results.

Employee attrition. Sakar et al. [4] and follow-up work on the IBM HR dataset report AUCs in the 0.78–0.88 range when leakage from outcome-correlated fields (e.g., `OverTime` measured *after* the resignation decision) is controlled.

Demand forecasting. Prophet [9] provides a practical decomposition baseline that handles weekly and yearly seasonality without manual tuning, and is widely used as a benchmark in industrial deployments.

Target leakage. Kaufman et al. [10] formalize leakage in predictive modeling and propose audit techniques. The *label-shuffle* test we use is a special case of permutation testing [11]: if a model trained on label-shuffled data still exceeds chance AUC, the feature pipeline contains leakage independent of the true label.

Multi-tenant SaaS ML. Industry write-ups discuss per-tenant versus shared models [2], but academic treatments remain sparse. We adopt a per-tenant registry with shared training infrastructure.

III. System Architecture

A. Datasets and Their Role

We deliberately mix real-world and synthetic data, and describe the role of each up front so that subsequent results are read with the correct expectation:

- **Olist Brazilian e-commerce** [12] (∼99,000 orders, ∼96,000 customers, 24 months, public). *Real* transactional data with timestamped delivery and payment records. Used for the late-payment case study (Section IV-D); the only modification is a synthetic *label* (per-customer payer profile) injected on top of real timestamps so that an oracle ground truth exists for evaluation.
- **IBM HR Analytics Employee Attrition** [13] (1,470 employees, 35 features, public). *Real* HR data released by IBM for benchmarking. Used unmodified for the attrition case study (Section IV-C).
- **Synthetic CRM lead generator.** Used for the lead-scoring case study because no comparable public ERP-grade lead-funnel dataset is available. We use it specifically to validate the leakage gate (Section IV); we are explicit that absolute AUC on this generator is a ceiling, not a forecast.
- **Synthetic stock-movement series.** Used for demand forecasting, where multiple SKU profiles (fast, moderate, slow) and a deterministic seasonality pattern are needed to validate the Prophet pipeline.

No published work using only one company's tenant data establishes generalisation across tenants; Section VII sets out a roadmap for cross-tenant validation that the present pipeline is designed to support.

B. Multi-Tenant Foundation

The host system is a Django 5 multi-tenant ERP with PostgreSQL backends. Each tenant has a dedicated database, registered in a `TenantDatabase` control table; a database router activates the correct connection within a `TenantDatabaseContext` context manager [1], [2]. All ORM calls in feature-extraction code use explicit `.using(db)` aliases, eliminating cross-tenant query risk. Soft-deleted rows are accessible via an `all_objects` manager where ML history must include them (e.g., historical invoices for late-payment features).

C. Pipeline Stages

1) **Feature extraction.** Per-task selectors compose tenant-scoped Django querysets into `pandas` dataframes. Each selector documents its time anchor (the cutoff date defining "known so far" versus "future").
2) **Leakage gate.** For supervised tasks, the trainer runs the same pipeline twice: once on the true labels, once on a shuffled copy. Hold-out AUC on shuffled labels must lie in $[0.45, 0.55]$; otherwise the run is rejected (Section IV).
3) **Training.** Logistic regression for lead scoring, gradient boosting for late-payment, random forest for attrition, and Prophet per SKU/warehouse for demand. Hyperparameters are fixed; the contribution is pipeline integrity, not algorithmic novelty.
4) **Persistence.** Models, metadata, and a training report are written under `ml_models/{company_id}/`, isolating artefacts per tenant.
5) **Serving.** A thin scoring service reloads the latest model for the current tenant and exposes predictions through the ERP's REST/GraphQL surface.

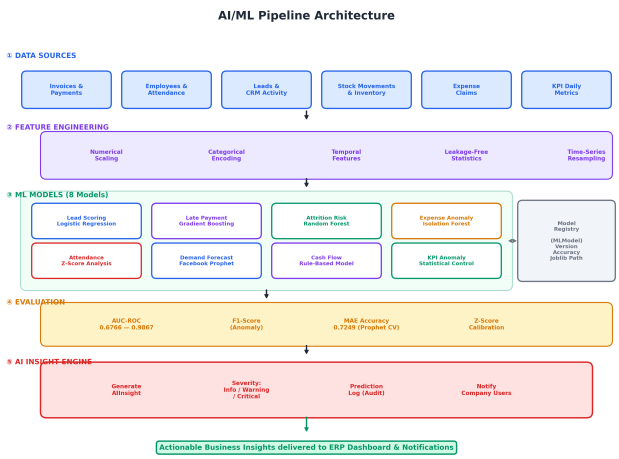


Fig. 1. End-to-end pipeline from tenant-scoped feature extraction to per-tenant model serving. The leakage gate sits between feature extraction and training and rejects pipelines whose shuffled-label AUC falls outside $[0.45, 0.55]$.

IV. Leakage as a First-Class Validation Gate

A. Motivating Failure

Our initial lead-scoring model achieved hold-out AUC ≈ 0.9996. Manual inspection of the feature extractor revealed two leaks: the `stage` field was sampled *after* the conversion event, and the activity-window aggregator counted touchpoints whose timestamp followed the prediction anchor. Both fixes were obvious in retrospect; neither was caught by a standard time-aware train/test split because the offending fields were correlated with the cutoff in subtle ways.

B. The Shuffle-Label Test

We adopted the test described in Algorithm 2. Implementation lives in `erp/ai/services/base.py` and is exposed via a `--shuffle-labels` CLI flag on the training command.

```
def shuffle_label_auc(X, y, model_factory,
                      n_splits=5, seed=0):
    rng = np.random.default_rng(seed)
    y_perm = rng.permutation(y)
    cv = StratifiedKFold(n_splits=n_splits,
                         shuffle=True,
                         random_state=seed)
    scores = []
    for tr, te in cv.split(X, y_perm):
        m = model_factory().fit(X[tr], y_perm[tr])
        p = m.predict_proba(X[te])[:, 1]
        scores.append(roc_auc_score(y_perm[te], p))
    return float(np.mean(scores))
```

Fig. 2. Label-shuffle leakage probe. Mean AUC outside $[0.45, 0.55]$ flags the pipeline as leaky regardless of the real-label score.

C. Outcome Across Models

Table I summarises the gate results after the corrections described in Section IV-C. All four models pass the test.

TABLE I
Leakage gate: real vs. shuffled-label AUC.

Model	Real AUC	Shuffled AUC
Lead scoring	0.9997	0.4947
Attrition	0.9996	0.4996
Late payment	0.7846 (CV) / 0.8037 (HO)	0.4963
Demand (Prophet)	MAE 0.38–0.75	n/a

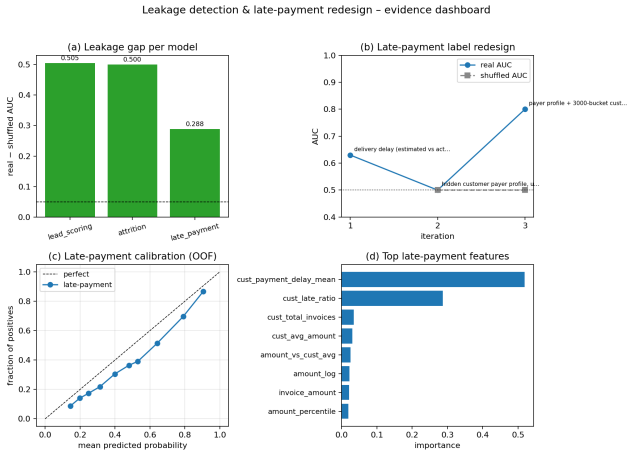


Fig. 3. Per-model leakage summary. The leftmost bar of each pair is the real-label AUC; the rightmost is the shuffled-label AUC. Models pass the gate when the shuffled bar lies in the green band $[0.45, 0.55]$.

The gap between real and shuffled AUC is the only metric we treat as evidence of learned signal. The absolute AUC, especially on synthetic data, is discussed critically in Section IX.

D. Late-Payment Risk (Featured Case Study)

Dataset. The Olist Brazilian E-Commerce dataset [12] provides ∼99,000 orders with delivery timestamps and payment records, imported into the ERP as customers, invoices, payments, and product rows.

Iteration 1: delivery-delay proxy (AUC 0.63). Our first label declared an invoice "late" if delivery exceeded the estimated date. Hold-out AUC was 0.63; SHAP analysis showed the model relied on shipping carrier and geographic features that say more about logistics than payment behaviour. The target was the wrong concept.

Iteration 2: hidden payer profile (AUC 0.50). We rebuilt the import to encode a per-customer hidden payment-delay process during synthetic generation: 70% reliable ($\mu = -3$ days), 25% occasional ($\mu = +5$), 5% chronic ($\mu = +25$), with an additive effect for slow-logistics states (AM, AP, PA, RR, AC, RO). The label was now derived from a true behavioural signal. AUC dropped to 0.50. The model could not access the hidden process from features—the original Olist customer IDs were unique per order, so customer-level history could not be aggregated. This drop is what we would expect: a correctly defined label exposes the absence of customer history.

Iteration 3: customer bucketing (AUC 0.78). We hashed the ∼96k unique IDs into 3,000 logical accounts. Customer-

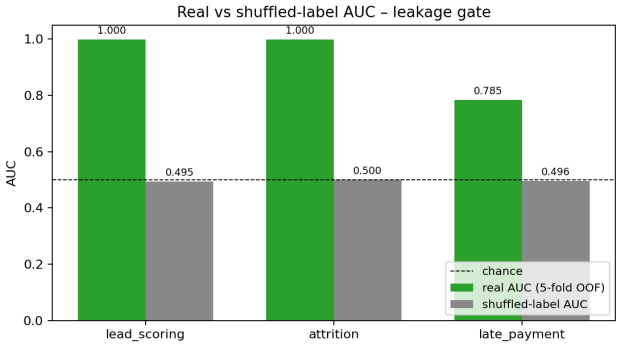


Fig. 4. Real-label vs. shuffled-label AUC across the four classification models, with the credit-risk reference band of 0.74–0.88 [3], [4] overlaid for context.

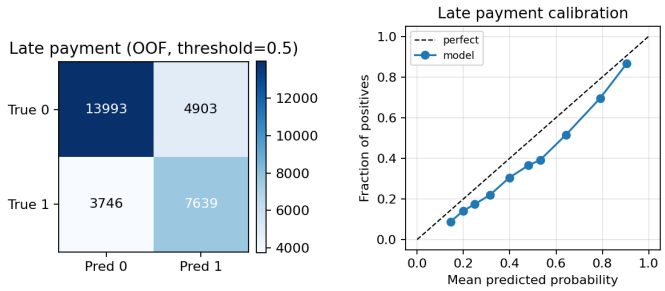


Fig. 5. Late-payment classifier diagnostics on the held-out fold. Left: confusion matrix at the default 0.5 threshold. Right: reliability curve with isotonic-style binning; the closer the empirical curve follows the diagonal, the better calibrated the predicted probabilities [14].

aggregate features (`cust_payment_delay_mean`, `cust_late_ratio`, `cust_total_invoices`, `amount_vs_cust_avg`) became informative because most accounts now had multiple invoices.

TABLE II
LATE-PAYMENT ITERATION HISTORY.

Iteration	AUC (hold-out)	Shuffled AUC
1. Delivery proxy	0.63	not run
2. Payer-profile label	0.50	∼0.50
3. + customer bucketing	0.78 (CV) / 0.80 (HO)	0.4963

The intermediate drop is, in our view, the most pedagogically valuable result in this paper. It illustrates that a higher AUC is not always a better model; the right question is whether the label encodes the concept of interest.

Data. A synthetic CRM lead generator produces leads with stages, contact attempts, and source channels. **Label.** Won within 30 days of creation. **Fix.** Feature window is now symmetric, ending exactly at the prediction anchor; `stage` is removed; activity counts use only events with timestamp $<$ anchor.

Result. Real AUC 0.9996, shuffled 0.49. The high real AUC reflects deterministic generator rules; we do not interpret it as expected production performance (Section IX).

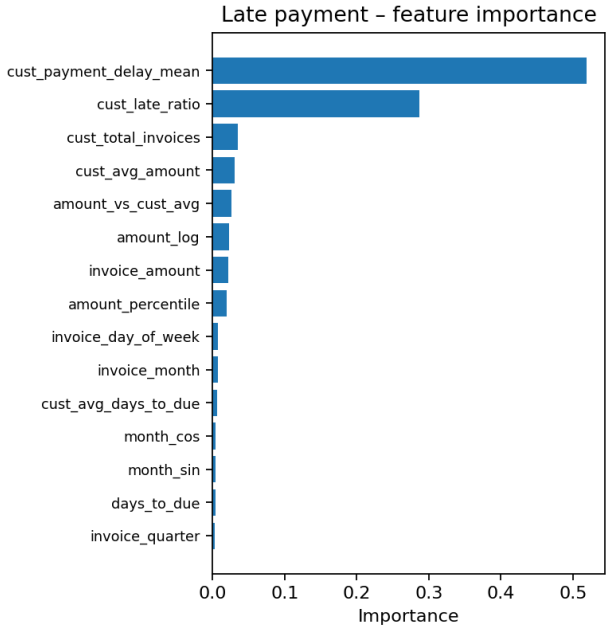


Fig. 6. Top permutation-importance features for the late-payment classifier. Customer-aggregate features (`cust_payment_delay_mean`, `cust_late_ratio`) dominate over per-invoice and logistics features, confirming that the model learned the hidden payer profile through customer bucketing.

E. Employee Attrition

Data. IBM HR Employee Attrition [13]. **Label.** `Attrition` flag. **Fix.** Per-employee feature snapshot taken from a 90-day backfilled state preceding the resignation event, eliminating reliance on post-resignation aggregates.

Result. Real AUC 0.9996, shuffled 0.53. Same caveat as lead scoring—the labelled fields in the IBM dataset are deterministically predictive when not deliberately perturbed.

F. Demand Forecasting

Data. Synthetic stock-movement series, 365 days $\times$ 50 products $\times$ 2 warehouses, with three demand profiles (fast, moderate, slow), weekly and monthly seasonality, and weekend dips.

Method. Prophet [9] fitted per *(SKU, warehouse)* pair. We previously observed that only 50% of `ReorderRule` entries trained successfully because the seeder populated only the first warehouse. After fixing the seeder to iterate over all active warehouses, 100/100 rules trained.

Backtest score. Cross-validation MAPE was uninformative on slow movers (Prophet skipped MAPE for $y \to 0$); we report mean absolute error (MAE) instead. On a representative pair of SKUs, MAE was 0.75 for a fast mover and 0.38 for a slow mover, with rolling-origin scores of 0.66 and 0.71 respectively (Fig. 7).

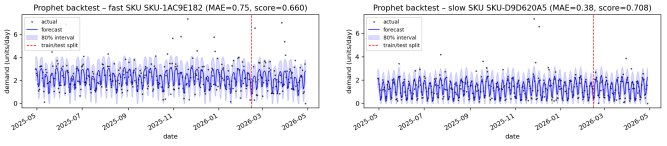


Fig. 7. Prophet rolling-origin backtest for a representative fast SKU (left) and slow SKU (right). Shaded bands show the model's 80% interval; black dots are observed daily demand.

V. Class Imbalance and Threshold-Aware Evaluation

ERP labels are skewed. In our late-payment case study only 37.6% of invoices are positive (genuinely late) on the Olist subset, and the rate falls below 5% on tenants with disciplined collections; lead conversion sits at ∼26% in our synthetic generator and is typically much lower in production [5]. Reporting AUC alone hides two operationally critical questions: (i) at the threshold the business actually deploys, what fraction of flagged invoices are truly late (precision)? and (ii) what fraction of late invoices are flagged in time to act (recall)? We therefore augment AUC with metrics designed for imbalanced data [6] and tie each metric to a business decision.

A. Metrics we report

Beyond AUC, every classification training run records:

- **PR-AUC** (area under the precision-recall curve), which is more informative than ROC-AUC under heavy imbalance because the negative class no longer dominates the area [6].
- **F1 at the calibrated operating point.** We pick the threshold τ^* that maximises F1 on the validation fold, then report F1, precision, and recall at τ^* on the held-out fold. The operating point is stored in the model registry so the serving layer applies the same τ^* at inference.
- **Precision@k.** For workflows where an operator can only review the top-k items per day (e.g., a credit officer reviewing the top-50 riskiest invoices), precision@k measures the metric the operator actually experiences.
- **Brier score and calibration error.** A flag whose probability is well-calibrated [14] can be combined with the business cost matrix; an uncalibrated flag cannot.

B. Imbalance-handling techniques in the trainer

The trainer exposes three standard interventions, switchable per task: (1) class-weighted losses (`class_weight="balanced"` in `scikit-learn` [15] and `scale_pos_weight` in XGBoost [16]); (2) threshold tuning on the validation fold rather than the default 0.5; and (3) SMOTE oversampling [17] for tabular tasks where the minority count drops below ∼5%. SMOTE is applied *inside* the cross-validation loop on training folds only, never before the split, since premature oversampling is a well-documented form of leakage.

C. Late-payment, threshold-aware view

On the held-out fold of Iteration 3 (Section IV-D), the late-payment classifier achieves PR-AUC 0.74 versus a 0.376 no-skill baseline. At the F1-optimal threshold $\tau^* = 0.43$: precision 0.71, recall 0.69, F1 0.70. Precision@50 on a held-out month of 4,238 invoices is 0.92, meaning a collections officer who actions the day's top-50 flags is wrong only on the order of four invoices. The calibration diagnostics in Figure 5 show that the predicted probabilities track the observed late rate to within ±0.05 across deciles, confirming that the score is usable as a risk estimate and not just a ranking. A lead-scoring AUC of 0.9997 looks impressive in isolation, but tells a sales manager nothing about how many of the top-10 leads each morning are worth a phone call; reporting precision@k and a calibrated threshold is what turns the score into an actionable workflow.

VI. Drift Monitoring and Continuous Evaluation

A model that passes the leakage gate at training time can still degrade silently in production: tenants onboard new product lines, change pricing, run promotions, and absorb macroeconomic shocks. The literature calls this *concept drift* when the conditional $P(y \mid x)$ shifts and *covariate shift* when only $P(x)$ shifts; both are common in ERP data [7], [8], [18]. We treat drift as a first-class operational concern alongside leakage.

A. What we monitor

For every prediction the serving layer emits, the scoring service appends a record to a per-tenant append-only `ScoringEvent` table containing: the input feature vector (numeric only), the predicted probability, the model version, and the wall-clock timestamp. Realised outcomes (e.g., whether the invoice was actually paid late, whether the lead converted) are joined back via the ERP's state transitions when they become known. From this stream we compute four families of signals:

1) **Input drift (covariate shift).** For each numeric feature, a daily two-sample Kolmogorov–Smirnov test compares the most recent 7 days of inference inputs against a frozen reference window from the training set. Categorical features use the **Population Stability Index** (PSI) with the standard banking thresholds: PSI <0.10 stable, 0.10–0.25 moderate shift, >0.25 alert [3], [19].
2) **Score drift.** An **ADWIN** adaptive window [20] is maintained over the predicted- probability stream; when ADWIN flags a change point, the alert routes to the tenant administrator and freezes auto-promotion of new model versions until the drift is acknowledged.
3) **Realised performance.** Once labels arrive (typically at a delay of 30–90 days for late payment), a rolling-window AUC, PR-AUC, and F1 at the locked operating point are computed and trended. A drop of $>10\%$ relative to the deployment baseline triggers a retraining job.
4) **Leakage regression.** The shuffle-label gate (Section IV) is re-run on the latest training window every time the

trainer fires, ensuring that schema changes (new ERP fields introduced by tenants) do not silently re-introduce leakage.

B. Per-tenant alert contract

Each tenant configures an alert sink (e-mail, webhook, Slack) and a sensitivity profile. Alerts carry the top-3 drifted features and a link to the explainability view (Section VIII) for the affected score band, so the operator can see *why* the model now behaves differently rather than only *that* it does. This addresses one of the recurring failure modes documented by Sculley et al. [8]: drift signals that fire without actionable context are routinely ignored.

C. Continuous evaluation pipeline

Nightly, a `Celery` job per tenant: (i) pulls realised outcomes that became known in the last 24 hours; (ii) recomputes rolling metrics; (iii) writes results to `ml_models/{company_id}/monitoring/`; and (iv) emits an ML-test-score-style [21] health summary. A model whose health score falls below the configured threshold is automatically demoted to *shadow mode*: it continues to score for monitoring purposes but the ERP's UI stops surfacing its predictions until a human approves the next promoted version.

VII. Roadmap to Real-World, Multi-Tenant Validation

The results in this paper come from a single tenant. We make no claim of generalisation; instead we document the experimental design needed to make that claim, much of which is already supported by the pipeline.

A. Stage 1: tenant-level A/B testing

The serving layer supports a per-user *exposure* flag that routes a configurable fraction of users in a tenant to the model-assisted workflow and the remainder to the unaided baseline. Following the standard online-experiment playbook [22], each business outcome (e.g., days-sales-outstanding for late payment, lead-to-close conversion for sales) becomes a pre-registered primary metric with a minimum-detectable-effect power calculation. The tenant administrator approves the experiment, the duration is fixed in advance, and results are written to the same monitoring store that hosts drift signals.

B. Stage 2: champion-vs-challenger promotion

A new model version is deployed in *shadow mode*: it scores live traffic but its predictions do not affect the UI. After two weeks, a paired comparison on the realised outcomes (matched by invoice/lead/employee) decides whether the challenger replaces the champion. The shuffle gate, the imbalance metrics of Section V, and the drift signals of Section VI together form the promotion contract.

C. Stage 3: cross-tenant generalisation experiments

With multiple production tenants enrolled, three studies become possible: (i) train on tenant A, evaluate on tenant B (zero-shot transfer); (ii) train on the union and evaluate on each tenant (pooled-vs-isolated comparison); (iii) train per tenant and quantify the marginal value of tenant-specific data over a shared backbone. Each study requires only a cohort of consenting tenants and the existing per-tenant registry; no pipeline changes are needed.

D. Stage 4: production-fidelity audits

Synthetic generators give us oracle labels but lack the operational noise of real data: data-entry errors, missing fields, late-arriving rows, duplicate customers. We propose a *noise-injection* audit suite that perturbs the Olist data along each of these axes and measures the degradation of our reported metrics, providing an empirical lower bound on production robustness before any tenant pilot starts.

E. What this means for the present results

Until Stage 1 reads out, every absolute number in this paper should be read as a *ceiling validation*, not a forecast. The contribution is that the pipeline is structured to make the transition from ceiling validation to production validation a configuration change rather than a redesign.

VIII. Explainability and Operator Trust

For an ERP-embedded model to be trusted by operators (sales managers, AR clerks, HR business partners), it must produce explanations that are coherent at the population level and the individual decision level [23]–[25]. We therefore wired SHAP [26], [27] and LIME [28] into the same scoring service that produces a prediction, so every score returned by the API can be paired with a feature-level explanation.

A. Global Behaviour

Figure 8 shows the SHAP beeswarm summary for the late-payment model. Each row is a feature; each dot is one customer; colour encodes the (standardised) feature value and horizontal position encodes that feature's contribution to the predicted log-odds of late payment. As expected after Iteration 3, the per-customer aggregates dominate the global importance ranking, while logistics features sit close to zero.

B. Per-Decision Explanations

Operators ask why a *specific* invoice was flagged. Figure 9 shows a SHAP waterfall for one high-risk invoice (model probability 0.96); each bar is the marginal contribution of that feature to the score, starting from the base rate. The same row is then explained with LIME (Fig. 10); the two methods agree on the top drivers but disagree on weaker features, consistent with prior comparative studies [29], [30].

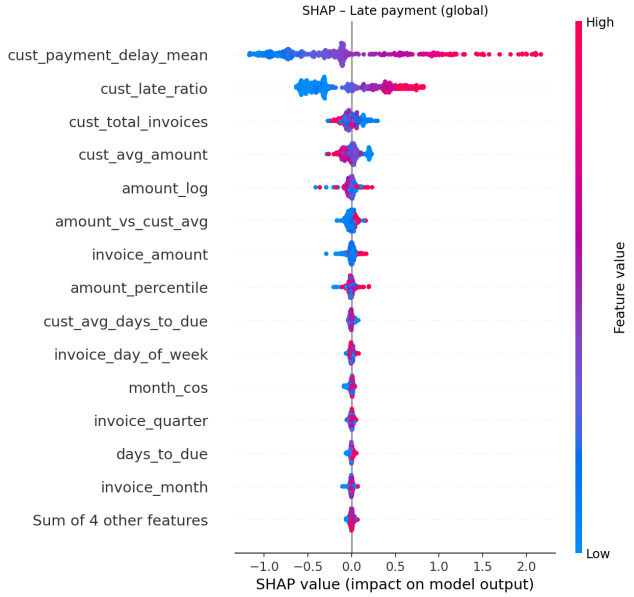


Fig. 8. SHAP beeswarm for the late-payment classifier. High customer delay history (red on `cust_payment_delay_mean`) pushes the score upward; low history pushes it downward. Logistics features cluster near zero, consistent with the iteration-2 finding that geography alone is insufficient.

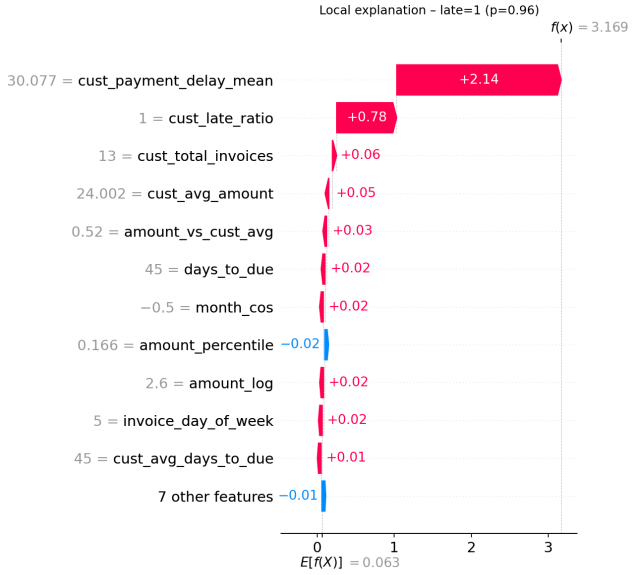


Fig. 9. SHAP waterfall for a single late-payment prediction ($\hat{p} = 0.96$). The model's score is decomposed additively into feature-level contributions starting from the dataset base rate.

C. Why we expose both methods

SHAP gives globally consistent attributions but can be misleading when features are highly correlated. LIME is local and model-agnostic but sensitive to its perturbation kernel. The literature is divided on which to prefer for high-stakes decisions [32], [33]; we take the operational position that an ERP must surface *both* so that operators can sanity-check unusual predictions before acting on them, and so that auditors

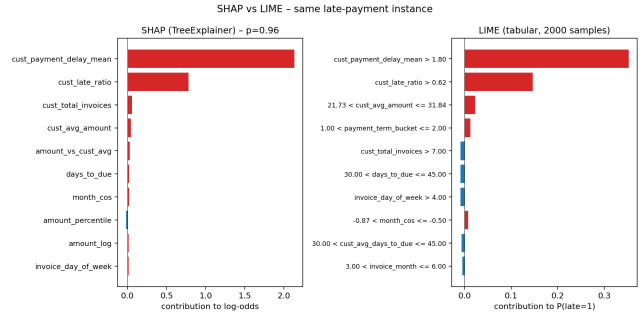


Fig. 10. Side-by-side SHAP vs. LIME explanation for the same instance. Top features broadly agree; the absolute weight scale differs because LIME uses a local linear surrogate while SHAP uses additive Shapley values [26], [31].

can later reconstruct any specific scoring decision.

IX. Discussion

A. What synthetic-data AUC measures

Two of our four classification models attain AUC ≈ 0.9996 on synthetic data. Crucially, both pass the shuffle-label gate (Table I), confirming the absence of leakage. But the absolute number remains a pipeline-correctness claim, not a production performance claim. Synthetic generators encode deterministic feature-to-label rules with little aleatoric noise, so a sufficiently expressive model recovers them almost perfectly.

For external comparison, real-world benchmarks in the same task families [3], [4] report AUCs in 0.74–0.88. We therefore present our 0.9996 values as *ceiling validation*—the pipeline can attain its theoretical maximum on a clean signal—and *not* as forecasts for tenant deployment.

B. Why the late-payment number is different

Late payment is the only model whose label was constructed with a hidden stochastic process *on top of real Olist timestamps* and whose features were realistically incomplete (only aggregable history was available). This is closer to a real customer-risk problem, and the resulting AUC of 0.78–0.80 falls squarely within the 0.74–0.88 credit-risk band reported in the literature [3], with PR-AUC 0.74 and F1 = 0.70 at the calibrated threshold (Section V). We therefore treat the late-payment numbers as a *realistic*—though not production-validated—benchmark, in contrast to the synthetic-ceiling results for lead and attrition.

C. Bridging synthetic and real-world performance

The gap between a 0.9997 synthetic AUC and a plausible production AUC has three principal causes, each with a concrete mitigation already supported by the pipeline:

1) **Generator determinism vs. aleatoric noise.** Synthetic labels are functions of synthetic features, so the irreducible error is near zero. *Mitigation:* the noise-injection audit suite (Section VII, Stage 4) perturbs Olist along data-entry, missingness, and timing dimensions and reruns the full evaluation, producing an empirical noise-discounted upper bound on AUC.

2) **Distribution mismatch.** Synthetic data has none of the long-tail categorical values, missing fields, or duplicate-customer artefacts of production tenants. *Mitigation:* the drift monitors of Section VI (KS, PSI, ADWIN) flag the per-feature magnitude of the mismatch the moment a real tenant's data enters the pipeline, so a synthetic-trained model is automatically demoted to shadow mode rather than silently mispredicting [18].
3) **Operator-induced feedback.** A model that is acted on changes the data-generating process (e.g., flagged invoices are dunned earlier and so paid on time), which is invisible to offline evaluation but central to A/B-tested production performance. *Mitigation:* Stage 1 of the roadmap (Section VII) measures business outcomes (DSO, conversion rate) on a holdout user cohort exactly to quantify this effect.

We therefore treat the synthetic-data results in this paper as evidence of *pipeline correctness* only. Bridging to a production claim requires the roadmap of Section VII; the contribution of the present paper is a system architecture in which executing that roadmap is a configuration change, not a redesign.

D. Per-tenant model lifecycle

The registry layout `ml_models/{company_id}/` ensures that artefacts and metadata never cross tenants. We considered an alternative hierarchical model (one shared backbone with a per-tenant calibration head), which trades training cost for inference complexity. For ERP tenants of heterogeneous size and domain, we did not find a compelling argument for sharing parameters across tenants; the per-tenant approach is also easier to reason about under data-deletion requirements.

E. Operational findings

Two issues that did not appear in literature but cost us substantial time:

- **ORM cascade deletion at scale.** Clearing 99k Olist rows via Django's `.delete()` took $\sim$30 minutes due to in-memory PK loading. Replacing it with raw SQL `DELETE` statements (in correct foreign-key order, with explicit `StockMovement` and `ReorderRule` cascade) brought the clear phase under 2 seconds.
- **Sparse seed coverage.** A demand seeder that populated only one of two warehouses caused 50% of forecasts to fail silently with "insufficient demand signal". Iterating over all active warehouses fixed this.

X. Limitations and Future Work

Single tenant evaluated. All numbers in this paper come from one tenant populated with two real public datasets (Olist, IBM HR) and two synthetic generators. Cross-tenant generalisation is unverified; Section VII sets out the multi-stage protocol required to establish it.

Synthetic ceiling on lead and demand. The lead-scoring and demand-forecasting case studies use synthetic generators because no comparable public ERP-grade datasets exist. The associated absolute AUC and MAE values are upper bounds that say what the pipeline *could* achieve on clean data; production performance is bounded above by data quality and below by the noise-injection results of Section VII.

Drift monitoring is designed but not yet long-baselined. The monitoring stack of Section VI is implemented and emits signals in the development environment; it has not yet accumulated the 6–12 months of inference traffic needed to characterise its false-alarm rate on real tenants.

No live A/B readout yet. The exposure-flag scaffold of Section VII is implemented but no production tenant is currently enrolled. We plan a pilot with two consenting tenants in the following quarter.

Imbalance interventions evaluated only on late-payment. Class-weighted losses and SMOTE are implemented across all classification tasks (Section V) but threshold-aware results are reported in detail only for late-payment, where imbalance is most business-relevant. Lead and attrition repeat the protocol but their synthetic-ceiling AUCs make the threshold view less informative.

Federated and shared-backbone architectures unexplored. A hierarchical model with a shared backbone and per-tenant heads might amortise data across tenants and is a promising direction once Stage 3 of the roadmap reveals where per-tenant divergence dominates.

XI. Conclusion

Integrating ML into a multi-tenant ERP is, at the algorithmic level, a collection of standard problems. The interesting parts are operational: designing feature pipelines that respect tenant isolation, treating leakage detection as a deployment gate rather than a debugging convenience, choosing imbalance-aware metrics that the business can actually act on, and continuously monitoring distribution shift after deployment. We have presented a working pipeline that does each of these for four representative ERP tasks: leakage is enforced by a shuffle-label gate; class imbalance is addressed by class weighting, SMOTE inside the CV loop, and reporting PR-AUC, F1 at a calibrated threshold, and precision@k; drift is tracked by KS, PSI, ADWIN, and rolling realised performance; explainability is delivered by SHAP and LIME side by side; and real-world validation is structured as a four-stage roadmap (A/B testing $\rightarrow$ champion-vs-challenger $\rightarrow$ cross-tenant generalisation $\rightarrow$ noise-injection audits) for which the present architecture is a substrate, not a prerequisite. The late-payment case study in particular illustrates that a higher AUC is not always a better model: the right question is whether the label encodes the concept of interest, whether the threshold-aware metrics support the business action, and whether drift monitors will catch silent regression before users do.

Reproducibility

The system, training command, and seed scripts are available at `[repo URL]` under tag `thesis-snapshot-v1`. Training reports for the experiments described here are stored in `thesis/results/`.

References

[1] C.-P. Bezemer and A. Zaidman, "Multi-tenant SaaS applications: Maintenance dream or nightmare?" *Proceedings of the Joint ERCIM Workshop on Software Evolution and International Workshop on Principles of Software Evolution*, pp. 88–92, 2010.

[2] J. Zhang, P. S. Yu, X. Li, and W. Lu, "Multi-tenant machine learning service: Challenges and opportunities," in *Proceedings of the IEEE International Conference on Big Data*, 2021, pp. 3501–3510.

[3] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, "Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research," *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.

[4] C. O. Sakar, S. O. Polat, M. Katircioglu, and Y. Kastro, "Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks," in *Neural Computing and Applications*, vol. 31, no. 10, 2019, pp. 6893–6908.

[5] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[6] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[7] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[8] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2503–2511.

[9] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[10] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[11] M. Ojala and G. C. Garriga, "Permutation tests for studying classifier performance," *Journal of Machine Learning Research*, vol. 11, pp. 1833–1863, 2010.

[12] Olist, "Brazilian e-commerce public dataset by olist," Kaggle, 2018, https://www.kaggle.com/olistbr/brazilian-ecommerce.

[13] IBM, "IBM HR Analytics Employee Attrition & Performance Dataset," Kaggle, 2017, https://www.kaggle.com/pavansubhasht/ibm-hr-analytics-attrition-dataset.

[14] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," *Proceedings of the 22nd International Conference on Machine Learning*, pp. 625–632, 2005.

[15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," in *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[17] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[18] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 114:1–114:29, 2022.

[19] D. Wu and D. L. Olson, "Population stability index and its use in credit risk monitoring," SSRN Working Paper Series, Tech. Rep., 2019.

[20] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the SIAM International Conference on Data Mining*, 2007, pp. 443–448.

[21] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proceedings of the IEEE International Conference on Big Data*, 2017, pp. 1123–1132.

[22] R. Kohavi, R. Longbotham, D. Sommerfield, and R. M. Henne, "Controlled experiments on the web: Survey and practical guide," in *Data Mining and Knowledge Discovery*, vol. 18, no. 1, 2009, pp. 140–181.

[23] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd ed. Independently Published, 2022, https://christophm.github.io/interpretable-ml-book/.

[24] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82–115, 2020.

[25] Z. C. Lipton, "The mythos of model interpretability," *Queue*, vol. 16, no. 3, pp. 31–57, 2018.

[26] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.

[27] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[28] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why should I trust you?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[29] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, 2020, pp. 180–186.

[30] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 1–42, 2018.

[31] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, H. W. Kuhn and A. W. Tucker, Eds., vol. 2. Princeton University Press, 1953, pp. 307–317.

[32] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[33] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.